PHASE 4 — CAPSTONE

Session 25

Capstone Kickoff

Project Charter · Team Formation · Capstone Architecture Review

Dr. ElSayed Mohamed Elsayed Baladoh

Tech Lead · Ph.D. · sayedbaladoh@yahoo.com

DAY	TIME
Wednesday	10:00 AM – 3:00 PM
DURATION	**MODE**
5 Hours	Offline — Anchor Day #4

Teams: 3 × 3 trainees · Build window: ≈ 2.5 weeks (S25 → S29)

Five Sessions. One Product, Built as a Team.

● LIVE

S25 Kickoff

S26 Gate G1

S27 Gate G2

S28 Gate G3

S29 Final

The five sessions are checkpoints ("gates"). The building itself happens in your own time between them — roughly 2 hours a day, per person.

The full 29-session roadmap lives in the Kickoff deck only — this strip shows Phase 4.

Session	You show live	You commit
S26 — G1	L1+L2: public GET, 401, ADMIN rules, Feign+resilience, idempotent payment	PRs merged w/ tests
S27 — G2	Saga + compensation; running in a K8s cluster; ArgoCD Synced	Dockerfiles, CI, Helm, manifests
S28 — G3	k6 results; one bottleneck found & fixed; trace across HTTP+Kafka	k6 scripts, Performance Report



PHASE 4 TRANSITION POINT

“Until today we learned patterns, one lab at a time. Some things were built twice — JwtAuthFilter became a Resource Server, Choreography became Orchestration, in-memory maps became databases.
That was the right way to LEARN. It is the wrong way to BUILD.”

“Today we start over — once — with everything we know. Each team builds the platform in its FINAL shape, from a clear blueprint, and defends every decision.”

THE PROBLEM — CS-01

A Team Without a Blueprint

Day 1	Three people, three IDEs, no design. "Writing docs wastes time."
Day 4	Order Service expects "available". Inventory returns "inStock". Nobody agreed on the contract.
Day 6	One member builds Choreography, another builds an Orchestrator — in the same service.
Day 10	The Kubernetes deploy fails. Every service reads config differently.
Day 14	The team rebuilds Order Service from scratch. The demo has ONE service that works.

Root cause: not a lack of skill — a lack of shared picture.

Lab Era vs Capstone

Aspect	Lab Era (Sessions 1–24)	Capstone (Sessions 25–29)
Repository	One personal repo, evolved lab by lab	One repository per team, from the Starter Repo
Security	JwtAuthFilter (S3), later Resource Server (S20)	Keycloak Resource Server from day one
Stock data	In-memory map (S6), later a DB	PostgreSQL + Flyway from day one
Saga	Choreography (S7), later Orchestration (S12)	ONE style, chosen by your team, justified in the ADD
Debt fixes	Outbox / Idempotency / Versioning added in S22	Built-in from the first commit
Guidance	One lab at a time	Full picture first: requirements, architecture, tech
Work mode	Individual labs	Team of 3, rotating roles, PR review, git log evidence

Vision & Product Definition

> *"Rebuild the platform we learned — once, in its final form — into a product a stranger can run, deploy, load-test and question."*

Product

- Enterprise E-Commerce Platform
- Customers browse a catalogue, place orders, and are notified
- Admins manage the catalogue and stock

Team Deliverable

- One independent Git repository per team
- 8 platform services + infra + K8s/Helm/GitOps
- Observability + load tests + docs + ONE Bonus Feature

FR-01 … FR-08 (Core)

ID	Requirement	Pri	Owner
FR-01	Anyone can browse products (paginated, public, no token)	M	Product, Gateway
FR-02	Admin can create, update, delete products (ADMIN only)	M	Product
FR-03	Product detail includes category name (read model)	S	Product
FR-04	Customers sign in via Keycloak; protected routes reject bad tokens	M	Gateway, Keycloak
FR-05	Order placed → API answers immediately: orderId + PENDING	M	Order
FR-06	Order checks stock sync before accepting; no stock = rejected before payment	M	Order, Inventory
FR-07	Stock reserved for the order; released if order fails	M	Inventory
FR-08	Payment processed exactly once — retried request never double-charges	M	Payment

FR-09 … FR-16

ID	Requirement	Pri	Owner
FR-09	CONFIRMED on payment success; CANCELLED + stock released on failure	M	Order, Inventory, Payment
FR-10	Read order status by id; list own orders only	M/S	Order
FR-11	Confirmation / cancellation notice; failed sends retried then DLT	M	Notification
FR-12	Admin can view and adjust stock levels	S	Inventory
FR-13	Public endpoints rate limited per client	S	Gateway (+Redis)
FR-14	Service-to-service calls use Client Credentials	C	Order, Inventory, Keycloak
FR-15	Product reads cached, evicted on every write (item + list)	S	Product, Redis
FR-16	Team delivers ONE Bonus Feature	M	Team choice

MoSCoW: M = Must (Core) · S = Should · C = Could. All Must + Should = complete Core product.

NFR-01 … NFR-10

ID	Requirement	Verified by
NFR-01	Payment down → orders stay PENDING or CANCELLED w/ compensation	Chaos test
NFR-02	GET /products P95<200ms (cached); POST /orders P95<800ms @20 VUs	k6 report (G3)
NFR-03	Gateway sustains ≥50 req/s on reads (or your justified target)	k6 report
NFR-04	No secrets in Git; JWT at Gateway; non-root; least-privilege SA	CI guards + review
NFR-05	No orphaned stock reservation >30s after CANCELLED	Test + query
NFR-06	One traceId across HTTP + Kafka hops	Zipkin screenshot
NFR-07	≥60% coverage; ≥1 TestContainers test per DB-owning service	JaCoCo in CI
NFR-08	docker compose up locally; helm install on Kubernetes	Live demo
NFR-09	Flyway migrations; every public API under /api/v1	Code review
NFR-10	At-least-once + idempotent consumers + DLT	Redelivery test

TARGET ARCHITECTURE

Eight Services, One Platform



Happy Path vs Failure Paths

Happy Path

- POST /api/v1/orders + JWT → Gateway validates
- Order: Feign stock check (CB/Retry/Bulkhead/TL)
- Order saves PENDING + outbox row (ONE transaction)
- Publisher sends OrderPlaced → Kafka
- Inventory reserves stock → InventoryReserved
- Payment charges once → PaymentCompleted
- Order → CONFIRMED · Notification sends email

Failure Paths

- No stock: REJECTED, payment never touched
- Payment fails: PaymentFailed → Inventory releases → InventoryReleased → Order CANCELLED
- Payment down: CB/retry fallback; order never lost
- Duplicate event: consumers check state first; Payment uses orderId as idempotency key
- Consumer keeps failing: @RetryableTopic → .DLT → alert log

TWO PATHS, TWO PURPOSES

Sync Path vs Async Path — One Flow

SYNC — Order → Inventory (Feign)

- Answers: "Can I accept this order RIGHT NOW?"
- The caller needs the answer to continue
- Protected by Resilience4j (Session 4-5 stack)

ASYNC — the Saga (Kafka)

- Carries the business transaction across services
- The caller does NOT wait for it to finish
- Choreography or Orchestration — your ADD decides

This is the Sync-vs-Async decision framework from Session 6, applied to one real flow: the stock pre-check must answer NOW; the Saga that confirms/cancels the order can take its time.

Three Levels: Clarity AND Freedom

FIX — **Fixed**

Not negotiable — the platform depends on it

No ADD needed

DEF — **Default**

Recommended choice; the team may replace it

Replacement needs a written justification (ADD §5–§7)

OPEN — **Open**

The team decides

Decision and reason recorded in the ADD

Layers 1–10

Layer	Lvl	Default	Alternative (justify in ADD)
Language / framework	FIX	Java 21, Spring Boot 3.x, Spring Cloud	—
Service discovery	DEF	Eureka (separate service)	Kubernetes DNS — explain client-side LB
Configuration	DEF	Config Server (separate service)	ConfigMap/Secret + env vars
Identity & security	FIX	Keycloak, Gateway = OAuth2 Resource Server	—
Data storage	FIX	PostgreSQL, DB-per-service, Flyway	—
Cache	DEF	Redis, cache-aside, evict on every write	Caffeine (single instance only)
Messaging	FIX	Kafka	—
Saga style	OPEN	Choreography (3 participants)	Orchestration — state location + trigger
Sync communication	DEF	OpenFeign	WebClient (reactive caller only)
Resilience	DEF	Resilience4j (CB/Retry/Bulkhead/TL)	Istio outlier detection, IN ADDITION

Layers 11–20

Layer	Lvl	Default	Alternative (justify in ADD)
Event publishing	DEF	Transactional Outbox + polling publisher	Direct publish + idempotent consumer
Read model	OPEN	Spring Data projection	Full separate read store (justify)
Testing	DEF	JUnit5, Mockito, @WebMvcTest, TestContainers	Pact, WireMock
Observability	DEF	Micrometer Tracing+Zipkin, Prometheus, JSON logs	—
Containers	FIX	Docker multi-stage, non-root, healthcheck	—
CI	DEF	GitHub Actions: test → build → push (sha tag)	—
Deployment	DEF	Kubernetes + Helm chart per service	Raw manifests (Helm ≥2 services)
GitOps	DEF	ArgoCD (automated, selfHeal, prune)	Helm-only — explain drift detection
Service mesh	OPEN	Not required	Istio mTLS — named problem only
Load testing / Not allowed	FIX	k6 (smoke→load→stress)	Elasticsearch, extra DBs/brokers, Axon

DESIGN CHOICE

Choreography vs Orchestration

3 participants, linear, no branching? → **Choreography (simpler, no coordinator)**

Need "where is order X" from one place? → **Orchestration (state in one table)**

Business rules will branch (fraud/loyalty)? → **Orchestration (rules live in one class)**

Team new to event-driven design? → **Choreography first, add tracing early**

Orchestrator state must survive a restart? → **Persist SagaState in PostgreSQL**

Different teams deploy on different schedules? → **Choreography (loose coupling)**

Cannot decide? → **Choreography + Outbox + idempotent consumers**

Choosing Orchestration "because it is more advanced" is not a reason — it fails ADD review.

ENGINEERING DECISION

One Monorepo per Team

- Team of 3, 8 services, shared parent POM, one CI, one demo → ONE monorepo per team (Starter Repo layout)
- Branching: short-lived branches (max 1–2 days), merged by Pull Request into main
- Never commit directly to main after Layer L0
- Every PR needs one review from a teammate (rotating Tech Lead decides on disagreement)

```
Commit convention:   capstone-Lx: short-description
  example:           capstone-L2: add-payment-idempotency

Pair work:            Co-authored-by: Name <email>
                      (keeps the git log honest for the Individual assessment)
```

Layers L0 → L6: Each Ends With Something That RUNS

Layer	Content	Definition of Done	Gate
L0	Repo, infra, Config+Eureka	verify-l0.sh passes	S25
L1	Product + Gateway + Keycloak	Public GET works; 401 no-token; /api/v1	G1—S26
L2	Inventory + Payment + Order (sync)	Feign+Resilience4j pre-check; idempotent pay	G1—S26
L3	Saga + Outbox + Notification	Happy path CONFIRMED; compensation; DLT	G2—S27
L4	Docker/CI/K8s/Helm/GitOps	ArgoCD Synced/Healthy; probes; Secrets	G2—S27
L5	Observability + k6	One trace HTTP+Kafka; bottleneck fixed	G3—S28
L6	Bonus + docs + demo	Bonus Core; final ADD; rehearsed demo	G4—S29

GATE STATUS RULE

Green / Amber / Red — and What to Cut

GREEN	The Layer's Definition of Done is demonstrated live.
AMBER	Partly done, with a written recovery plan before the next gate. Allowed once per team.
RED	No demonstrable result. 30-minute trainer intervention; apply the scope-cutting order.

If behind, cut in this order:

`Stretch goals → Bonus "Could" stories → FR-14 → FR-13 → FR-12 → reduce Bonus Core scope`

Never cut: tests, security rules, the compensation path, deployment.

Time Budget

Item	Estimate
Total team effort L1–L6	≈ 100–120 hours (≈ 35–40 h/trainee over 2.5 weeks, ~2h/day)
Heaviest layers	L3 (Saga+Outbox) and L4 (Kubernetes+GitOps) — start early
What the Starter Repo saves	≈ 20 hours of boilerplate: parent POM, skeletons, Dockerfile, Compose, CI, Keycloak realm, Prometheus config
Gate rhythm	Windows are uneven (~5, 2, 5, 2 days). Long windows = heavy layers; short windows = catch-up, tests, docs

YOUR STARTING POINT

Starter Repo — READY vs SKELETON vs EMPTY

READY

- config-server
- eureka-server
- docker-compose.yml
- Keycloak realm-export.json
- Prometheus scrape config
- verify-l0.sh

SKELETON

- api-gateway
- product-service
- order-service
- payment-service
- inventory-service
- notification-service

EMPTY + README

- deployment/kubernetes/
- deployment/helm/
- deployment/argocd/
- (you build these at L4 —
- the README explains the
- expected structure)

EACH TEAM OWNS ONE PRIMARY BONUS (FR-16)

Bonus Feature Cards

B1 — Product Reviews & Ratings

new service :8086

Rating 1–5 + text; avg rating via event; own DB + Helm + CI

B2 — Order Analytics Dashboard

CQRS + Grafana, no new infra

Read model from order events; 3-panel Grafana dashboard

B3 — Multi-Tenant Gateway

hardest conceptually

Tenant via JWT claim; data isolation; per-tenant rate limit

B4 — Real-Time Inventory Alerts

Kafka + SSE/WebSocket

LowStock event; admin alert within 2s; deduped per product

CAPSTONE = 35% OF THE COURSE GRADE

Grading Model

Team — 70%

- Architecture Quality (ADD, decisions, diagrams) — 25%
- Implementation + Tests (Core+Bonus, coverage, PRs) — 35%
- Deployment + Security (K8s, GitOps, hardening) — 20%
- Presentation (structure, demo, honesty) — 20%

Individual — 30%

- Oral Q&A in Session 29 — 60%
- (each member answers about ANY part of the system)
- Code contribution: git log across ≥3 services — 40%

A complete Core alone caps Implementation credit at 85% — the primary Bonus supplies the remaining 15%.

9 Trainees → 3 Teams of 3

- Skills Matrix (3 min): rate 1–5 on Spring Boot, Kafka/Saga, Docker/K8s/CI, Testing, Security/Keycloak — trainer spreads strong members across teams
- Trainees behind on labs pair with a strong partner and use the same Starter Repo — nobody starts from their old lab repo
- Roles rotate at every gate (G1→G2→G3→G4): Tech Lead · DevOps/Security · QA/Docs — everyone holds every role once
- Bonus draft (5 min): teams rank B1–B4; ties go to the lower Skills Matrix average, otherwise a coin flip
- Team Charter (one page, signed): working hours, channel, PR review SLA (24h), disagreement resolution

Five Questions, 8 Minutes Each

Q1	Database-per-service = no joins, no shared transactions. Which requirement feels this most?
Q2	Choreography or Orchestration — which would YOUR team choose, and what would make you switch?
Q3	Where exactly must idempotency exist, and what breaks if it is missing in each place?
Q4	Gateway forwards X-User-* headers. Should downstream services also validate the token? Cost of each choice?
Q5	At Gate G2 your team is 40% behind. What do you cut, what do you refuse to cut, how do you tell the trainer?

Lazy: "It is standard, so it is fine." Good: "The risk is X, our mitigation is Y." Excellent: "I would change Z, because…"

Design Before Code — Then Write the ADD

Min	Team activity
0–10	Create repo from template; add members + trainer; protect main; docker compose up
10–20	Draw on paper (CS-09): service boundaries, main-flow sequence, failure path
20–50	Write the ADD (8 sections): Problem · Bounded Context · API+Events · Data Model · Communication · Failure Modes · Security+Deploy · Test+Load Plan
50–65	Build the backlog: 12–15 stories, estimated, mapped to Layers/Gates, first owners assigned

Peer Review: circular A→B→C→A, 12 min per ADD + 3 min feedback. Result: Approved / Approved with changes / Rework.

PROJECT PROGRESS

What We Added Today

Capability Added

- Architecture-Approved Capstone Plan
- Every team knows what, in what shape, with what tech, and what "done" means

Services Updated

- No service code today
- 3 repos from Starter, L0 running, 3 peer-reviewed ADDs + backlogs

config	eureka	gateway	product	order	payment	inventory	notif.
L0	L0	L1	L1	L2/3	L2/3	L2/3	L3

Next: Sessions 26–28 are Gate reviews. Most building happens at home, ≈2 hours/day/person.



SESSION 25 COMPLETE

Capstone Blueprint Approved

Next Session

Session 26 — Gate G1 Review: Layers L1 + L2

Wednesday → Monday, 3:00–5:30 PM, Online

microservices-pro-course · capstone-team repositories · ITSharks